

JAVA PROGRAMMING LAB (BCS154)

B.Tech CSE - 1st SEMESTER

ROLL NO. BCS2026499

SUBMITTED TO:

MR. ABHAY GUPTA

SUBMITTED BY:

SUMIT GANGWAR

SN.	PROGRAM NAME	PAGE	DATE	SIGNATURE
1	Hello World, Variables and Arithmetic	3	26/10/2026	
2	Decisions and Loops with Numbers	6	26/10/2026	
3	Star and Number Patterns	8	26/10/2026	
4	Writing Functions	11	28/10/2026	
5	Arrays: Traversal and Counting	13	28/10/2026	
6	Arrays: Reverse and Duplicates	15	28/10/2026	
7	Counting Steps and Complexity	17	30/10/2026	
8	Matrices: Transpose and Row Sums	19	30/10/2026	
9	Matrices: Rows and Columns	21	30/10/2026	
10	Switch and ArrayList	23	02/11/2026	
11	Bubble Sort and Selection Sort	25	02/11/2026	
12	Insertion Sort	27	02/11/2026	
13	Strings: Methods and Palindromes	29	04/11/2026	
14	Characters: Counting and Positions	31	04/11/2026	
15	StringBuilder Methods	33	04/11/2026	

EXPERIMENT NO. 1

Hello World, Variables and Arithmetic

Student Name: _____ **Roll No.:** _____

1. Experiment Title

Hello World, Variables and Arithmetic

2. Aim

Print text, store values in variables and calculate with arithmetic operators.

3. Software / Tools Required

Java JDK; any Java IDE or text editor; terminal/command prompt.

4. Theory

Java programs begin execution in main(). Variables store values of specific data types such as int, double, char and String. Arithmetic operators +, -, *, / and % perform calculations. The final keyword declares a constant.

5. Algorithm / Procedure

Compile each Main.java file with javac and run it with java Main.

6. Program / Queries / Commands

Task 1: Hello World

Print two lines of text.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
        System.out.println("Welcome to Java");
    }
}
```

7. Input

No input; the text is fixed in the program.

8. Output



Task 2: Variables and data types

Create and display the specified variables.

```
public class Main {
    public static void main(String[] args) {
        int roll = 101;
        double percentage = 84.5;
        char section = 'B';
        String name = "Asha";
        System.out.println("Roll: " + roll);
        System.out.println("Percentage: " + percentage);
        System.out.println("Section: " + section);
        System.out.println("Name: " + name);
    }
}
```

7. Input

roll = 101, percentage = 84.5, section = B, name = Asha.

8. Output



Task 3: Arithmetic operations

Calculate sum, difference, product, quotient and remainder.

```
public class Main {
    public static void main(String[] args) {
        int a = 17, b = 5;
        System.out.println("Sum: " + (a + b));
        System.out.println("Difference: " + (a - b));
        System.out.println("Product: " + (a * b));
        System.out.println("Quotient: " + (a / b));
        System.out.println("Remainder: " + (a % b));
    }
}
```

7. Input

a = 17 and b = 5.

8. Output



Task 4: Circle area and perimeter

Use a constant PI to calculate the area and perimeter of a circle.

```
public class Main {
    public static void main(String[] args) {
        final double PI = 3.14;
        double radius = 7;
        System.out.println("Area: " + (PI * radius * radius));
        System.out.println("Perimeter: " + (2 * PI * radius));
    }
}
```

7. Input

PI = 3.14 and radius = 7.

8. Output



9. Result

The programs for Experiment 1 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.

EXPERIMENT NO. 2

Decisions and Loops with Numbers

Student Name: _____ **Roll No.:** _____

1. Experiment Title

Decisions and Loops with Numbers

2. Aim

Use if, else-if and loops to make decisions and repeat steps with whole numbers.

3. Software / Tools Required

Java JDK; Java IDE or text editor; terminal/command prompt.

4. Theory

Conditional statements select a path based on a boolean expression. The modulo operator checks divisibility. for loops repeat a known number of times. A leap year is divisible by 4 and not by 100, unless it is divisible by 400.

5. Algorithm / Procedure

Create Main.java, compile it, and run each task separately.

6. Program / Queries / Commands

Task 1: Even or odd

Use if-else and the modulo operator to classify n.

```
public class Main {
    public static void main(String[] args) {
        int n = 27;
        if (n % 2 == 0) System.out.println(n + " is even");
        else System.out.println(n + " is odd");
    }
}
```

7. Input

n = 27.

8. Output



Task 2: Grade using else-if

Display the grade based on the marks range.

```
public class Main {
    public static void main(String[] args) {
        int marks = 83;
        if (marks >= 90) System.out.println("Grade: A");
        else if (marks >= 75) System.out.println("Grade: B");
        else System.out.println("Grade: C");
    }
}
```

7. Input

marks = 83.

8. Output

```
OUTPUT | Java console

Grade: B
```

Task 3: Leap year check

Apply the leap-year divisibility rule.

```
public class Main {
    public static void main(String[] args) {
        int year = 2024;
        if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
            System.out.println("Leap year");
        else System.out.println("Not a leap year");
    }
}
```

7. Input

year = 2024.

8. Output

```
OUTPUT | Java console

Leap year
```

Task 4: Factorial using for loop

Multiply the integers from 1 through 5.

```
public class Main {
    public static void main(String[] args) {
        int factorial = 1;
        for (int i = 1; i <= 5; i++) factorial *= i;
        System.out.println("Factorial of 5 is " + factorial);
    }
}
```

7. Input

n = 5.

8. Output

```
OUTPUT | Java console

Factorial of 5 is 120
```

9. Result

The programs for Experiment 2 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.

EXPERIMENT NO. 3

Star and Number Patterns

Student Name: _____ **Roll No.:** _____

1. Experiment Title

Star and Number Patterns

2. Aim

Use a loop inside another loop to print stars and numbers in rows.

3. Software / Tools Required

Java JDK; Java IDE or text editor; terminal/command prompt.

4. Theory

Nested loops are used when a task has rows and columns. The outer loop controls the row, while the inner loop prints the symbols or values within each row.

5. Algorithm / Procedure

Run each pattern program and compare its console output with the pattern shown.

6. Program / Queries / Commands

Task 1: Square of stars

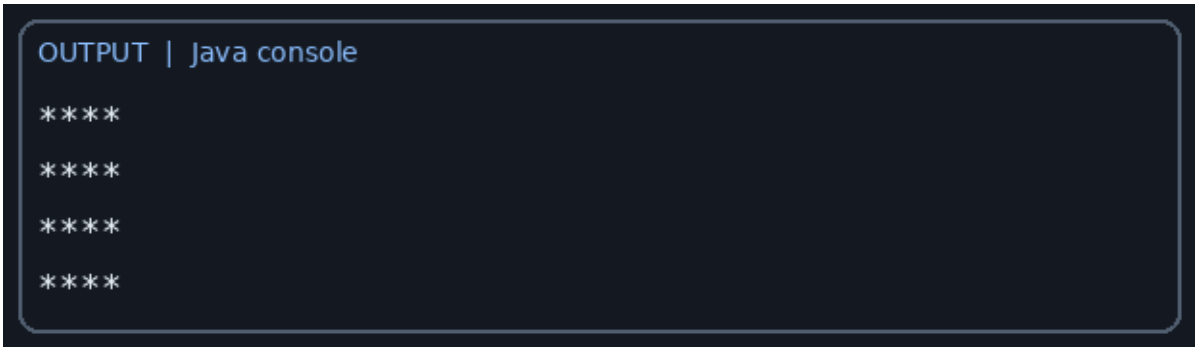
Print four rows, each containing four stars.

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= 4; j++) System.out.print("*");
            System.out.println();
        }
    }
}
```

7. Input

4 rows and 4 columns.

8. Output



Task 2: Right triangle of stars

Print rows containing one to four stars.

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= i; j++) System.out.print("*");
            System.out.println();
        }
    }
}
```

7. Input

4 rows.

8. Output



Task 3: Number triangle

Print increasing numbers from 1 up to the current row number.

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= i; j++) System.out.print(j + " ");
            System.out.println();
        }
    }
}
```

7. Input

4 rows.

8. Output



Task 4: Inverted triangle of stars

Print four stars in the first row, decreasing to one.

```
public class Main {
    public static void main(String[] args) {
        for (int i = 4; i >= 1; i--) {
            for (int j = 1; j <= i; j++) System.out.print("*");
            System.out.println();
        }
    }
}
```

7. Input

4 rows.

8. Output



9. Result

The programs for Experiment 3 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.

EXPERIMENT NO. 4

Writing Functions

Student Name: _____ **Roll No.:** _____

1. Experiment Title

Writing Functions

2. Aim

Write functions with parameters and return values, and call them from main.

3. Software / Tools Required

Java JDK; Java IDE or text editor; terminal/command prompt.

4. Theory

A method is a named block of code. Parameters pass values into a method, and a return statement sends a result back to the caller. Factorial is the product of positive integers up to n.

5. Algorithm / Procedure

Define the required methods outside main, call them from main, then compile and run.

6. Program / Queries / Commands

Task 1: Combination nCr

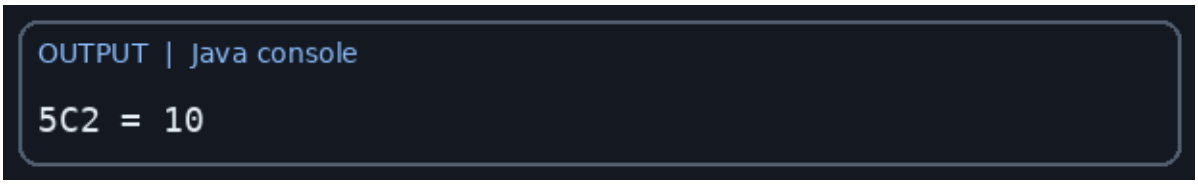
Use factorial to calculate the number of combinations.

```
public class Main {
    static int fact(int n) {
        int f = 1;
        for (int i = 1; i <= n; i++) f *= i;
        return f;
    }
    static int nCr(int n, int r) { return fact(n) / (fact(r) * fact(n - r)); }
    public static void main(String[] args) {
        System.out.println("5C2 = " + nCr(5, 2));
    }
}
```

7. Input

n = 5, r = 2.

8. Output



Task 2: Permutation nPr

Use factorial to calculate the number of permutations.

```
public class Main {
    static int fact(int n) {
        int f = 1;
        for (int i = 1; i <= n; i++) f *= i;
        return f;
    }
    static int nPr(int n, int r) { return fact(n) / fact(n - r); }
    public static void main(String[] args) {
        System.out.println("5P2 = " + nPr(5, 2));
    }
}
```

7. Input

n = 5, r = 2.

8. Output



Task 3: Larger of two numbers

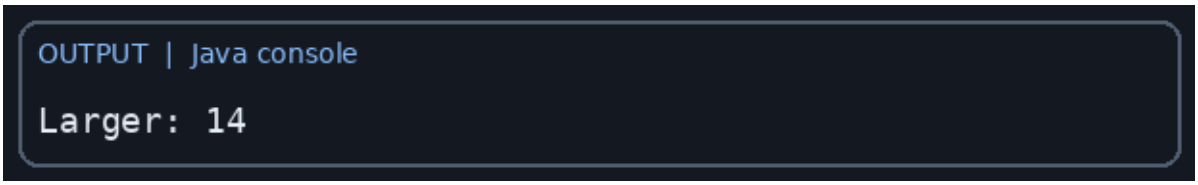
Return the larger of two integer values.

```
public class Main {
    static int maxOf(int a, int b) { return (a > b) ? a : b; }
    public static void main(String[] args) {
        System.out.println("Larger: " + maxOf(14, 9));
    }
}
```

7. Input

Values: 14 and 9.

8. Output



9. Result

The programs for Experiment 4 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.

EXPERIMENT NO. 5

Arrays: Traversal and Counting

Student Name: _____ **Roll No.:** _____

1. Experiment Title

Arrays: Traversal and Counting

2. Aim

Create an array, visit every element with a loop, and add up or count the values.

3. Software / Tools Required

Java JDK; Java IDE or text editor; terminal/command prompt.

4. Theory

An array stores a fixed number of values of the same type. Array indices begin at zero. The length field gives the number of elements. A loop can traverse an array to calculate totals, find maximum values or count values matching a condition.

5. Algorithm / Procedure

Declare the array, traverse it with a for loop, and display the requested result.

6. Program / Queries / Commands

Task 1: Array traversal

Print each element with its index.

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {3, 4, 5};
        for (int i = 0; i < arr.length; i++)
            System.out.println("Index " + i + ": " + arr[i]);
    }
}
```

7. Input

arr = {3, 4, 5}.

8. Output



Task 2: Sum and largest value

Find the sum and maximum element using a loop.

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {12, 45, 7, 33, 21};
        int sum = 0, largest = arr[0];
        for (int value : arr) {
            sum += value;
            if (value > largest) largest = value;
        }
        System.out.println("Sum: " + sum);
        System.out.println("Largest: " + largest);
    }
}
```

7. Input

arr = {12, 45, 7, 33, 21}.

8. Output



Task 3: Count values greater than 10

Count array elements whose value is greater than 10.

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 12, 2, 14, 7};
        int count = 0;
        for (int value : arr) if (value > 10) count++;
        System.out.println("Count greater than 10: " + count);
    }
}
```

7. Input

arr = {5, 12, 2, 14, 7}.

8. Output



9. Result

The programs for Experiment 5 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.

EXPERIMENT NO. 6

Arrays: Reverse and Duplicates

Student Name: _____ **Roll No.:** _____

1. Experiment Title

Arrays: Reverse and Duplicates

2. Aim

Print an array in reverse order, find the smallest value, and check an array for repeated values.

3. Software / Tools Required

Java JDK; Java IDE or text editor; terminal/command prompt.

4. Theory

An array can be traversed backwards by starting at its last index. A minimum search keeps track of the smallest value and its index. Nested loops can compare pairs of elements to detect duplicates.

5. Algorithm / Procedure

Use reverse traversal, a minimum-tracking loop, or pairwise comparisons as required.

6. Program / Queries / Commands

Task 1: Reverse array

Print the elements from last to first.

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        for (int i = arr.length - 1; i >= 0; i--)
            System.out.println(arr[i]);
    }
}
```

7. Input

arr = {10, 20, 30, 40, 50}.

8. Output



Task 2: Smallest value and index

Find the minimum value and the index where it occurs.

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {2, 5, 1, 8, 3, 4};
        int smallest = arr[0], index = 0;
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < smallest) { smallest = arr[i]; index = i; }
        }
        System.out.println("Smallest: " + smallest + " at index " + index);
    }
}
```

7. Input

arr = {2, 5, 1, 8, 3, 4}.

8. Output



Task 3: Detect repeated value

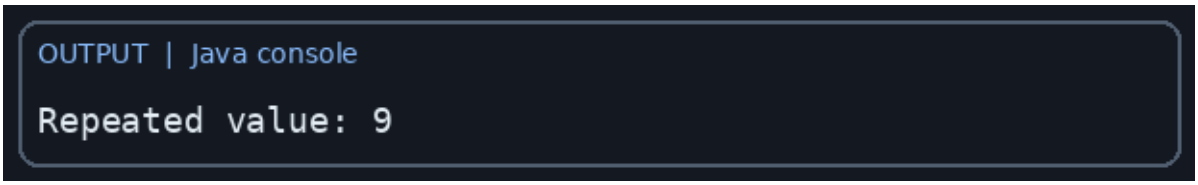
Use nested loops to find a repeated element.

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {4, 9, 2, 9, 7};
        boolean found = false;
        for (int i = 0; i < arr.length && !found; i++) {
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                    System.out.println("Repeated value: " + arr[i]);
                    found = true;
                    break;
                }
            }
        }
        if (!found) System.out.println("No repeated value");
    }
}
```

7. Input

arr = {4, 9, 2, 9, 7}.

8. Output



9. Result

The programs for Experiment 6 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.

EXPERIMENT NO. 7

Counting Steps and Complexity

Student Name: _____ **Roll No.:** _____

1. Experiment Title

Counting Steps and Complexity

2. Aim

Count the steps a program takes, and write its time complexity as a comment.

3. Software / Tools Required

Java JDK; Java IDE or text editor; terminal/command prompt.

4. Theory

Time complexity describes how an algorithm's running work grows with input size. A single loop is commonly O(n), two nested loops are O(n²), and repeatedly halving a value is O(log n). The programs count loop-body executions.

5. Algorithm / Procedure

Add a complexity comment above the loop or loops, increment a counter for each execution, and print the count.

6. Program / Queries / Commands

Task 1: Single loop steps

Count the executions of a loop from 1 to 10.

```
public class Main {
    public static void main(String[] args) {
        int steps = 0;
        // Time complexity: O(n)
        for (int i = 1; i <= 10; i++) steps++;
        System.out.println("Steps: " + steps);
    }
}
```

7. Input

Loop range: 1 through 10.

8. Output



Task 2: Nested loop steps

Count executions of the inner loop body.

```
public class Main {
    public static void main(String[] args) {
        int steps = 0;
        // Time complexity: O(n^2)
        for (int i = 1; i <= 10; i++)
            for (int j = 1; j <= 10; j++) steps++;
        System.out.println("Steps: " + steps);
    }
}
```

7. Input

Both loops run from 1 through 10.

8. Output



Task 3: Halving steps

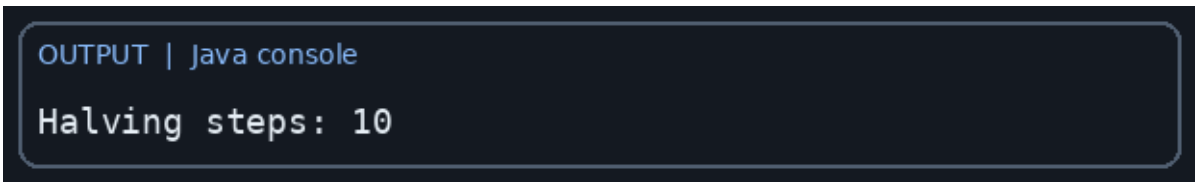
Repeatedly halve n until it becomes 1.

```
public class Main {
    public static void main(String[] args) {
        int n = 1024, steps = 0;
        // Time complexity: O(log n)
        while (n > 1) { n /= 2; steps++; }
        System.out.println("Halving steps: " + steps);
    }
}
```

7. Input

n = 1024.

8. Output



9. Result

The programs for Experiment 7 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.

EXPERIMENT NO. 8

Matrices: Transpose and Row Sums

Student Name: _____ **Roll No.:** _____

1. Experiment Title

Matrices: Transpose and Row Sums

2. Aim

Create two-dimensional arrays, visit them row by row, and build a new matrix from one.

3. Software / Tools Required

Java JDK; Java IDE or text editor; terminal/command prompt.

4. Theory

A two-dimensional array represents data in rows and columns. Nested loops access each cell using row and column indices. A transpose interchanges row and column positions. Row sums are calculated by adding values across each row.

5. Algorithm / Procedure

Use nested loops to fill, print, transpose or sum the specified matrices.

6. Program / Queries / Commands

Task 1: Create and print a 3 by 4 matrix

Fill the matrix with values 1 to 12, row by row.

```
public class Main {
    public static void main(String[] args) {
        int[][] m = new int[3][4];
        int value = 1;
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 4; j++) m[i][j] = value++;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 4; j++) System.out.print(m[i][j] + " ");
            System.out.println();
        }
    }
}
```

7. Input

3 rows and 4 columns; values 1 through 12.

8. Output

```
OUTPUT | Java console

1 2 3 4
5 6 7 8
9 10 11 12
```

Task 2: Matrix transpose

Create and print the transpose of the given 3 by 3 matrix.

```
public class Main {
    public static void main(String[] args) {
        int[][] m = {{1,2,3},{4,5,6},{7,8,9}};
        int[][] t = new int[3][3];
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++) t[j][i] = m[i][j];
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) System.out.print(t[i][j] + " ");
            System.out.println();
        }
```

```
    }
}
```

7. Input

m = {{1,2,3},{4,5,6},{7,8,9}}.

8. Output



Task 3: Sum of each row

Calculate and print the sum of every row.

```
public class Main {
    public static void main(String[] args) {
        int[][] m = {{1,2,3},{4,5,6},{7,8,9}};
        for (int i = 0; i < 3; i++) {
            int sum = 0;
            for (int j = 0; j < 3; j++) sum += m[i][j];
            System.out.println("Row " + (i + 1) + " sum: " + sum);
        }
    }
}
```

7. Input

m = {{1,2,3},{4,5,6},{7,8,9}}.

8. Output



9. Result

The programs for Experiment 8 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.

EXPERIMENT NO. 9

Matrices: Rows and Columns

Student Name: _____ **Roll No.:** _____

1. Experiment Title

Matrices: Rows and Columns

2. Aim

Visit the rows and columns of a matrix with nested loops to print, add and count its values.

3. Software / Tools Required

Java JDK; Java IDE or text editor; terminal/command prompt.

4. Theory

A matrix is a rectangular two-dimensional array. The first index identifies a row and the second identifies a column. Row and column operations use nested loops with the appropriate index held constant.

5. Algorithm / Procedure

Initialize the given matrix and use loops to access the requested rows, columns or values.

6. Program / Queries / Commands

Task 1: First and last row

Print the first row and last row of the matrix.

```
public class Main {
    public static void main(String[] args) {
        int[][] m = {{1,2,3},{4,5,6},{7,8,9}};
        System.out.print("First row: ");
        for (int j = 0; j < 3; j++) System.out.print(m[0][j] + " ");
        System.out.println();
        System.out.print("Last row: ");
        for (int j = 0; j < 3; j++) System.out.print(m[2][j] + " ");
        System.out.println();
    }
}
```

7. Input

m = {{1,2,3},{4,5,6},{7,8,9}}.

8. Output



Task 2: Sum of each column

Calculate and print the sum of each column.

```
public class Main {
    public static void main(String[] args) {
        int[][] m = {{1,2,3},{4,5,6},{7,8,9}};
        for (int j = 0; j < 3; j++) {
            int sum = 0;
            for (int i = 0; i < 3; i++) sum += m[i][j];
            System.out.println("Column " + (j + 1) + " sum: " + sum);
        }
    }
}
```

7. Input

m = {{1,2,3},{4,5,6},{7,8,9}}.

8. Output



Task 3: Count values greater than 5

Count all matrix values greater than 5.

```
public class Main {
    public static void main(String[] args) {
        int[][] m = {{1,2,3},{4,5,6},{7,8,9}};
        int count = 0;
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++) if (m[i][j] > 5) count++;
        System.out.println("Values greater than 5: " + count);
    }
}
```

7. Input

m = {{1,2,3},{4,5,6},{7,8,9}}.

8. Output



9. Result

The programs for Experiment 9 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.

EXPERIMENT NO. 10

Switch and ArrayList

Student Name: _____ **Roll No.:** _____

1. Experiment Title

Switch and ArrayList

2. Aim

Choose a branch with a switch statement, and store and change a list of numbers in an ArrayList.

3. Software / Tools Required

Java JDK; Java IDE or text editor; terminal/command prompt.

4. Theory

A switch statement selects one branch from several cases. ArrayList is a resizable collection from java.util. Its add, get, set, remove and size methods insert, access, update, delete and count elements.

5. Algorithm / Procedure

Import ArrayList where required, execute the switch or collection operations, and print the result.

6. Program / Queries / Commands

Task 1: Day using switch

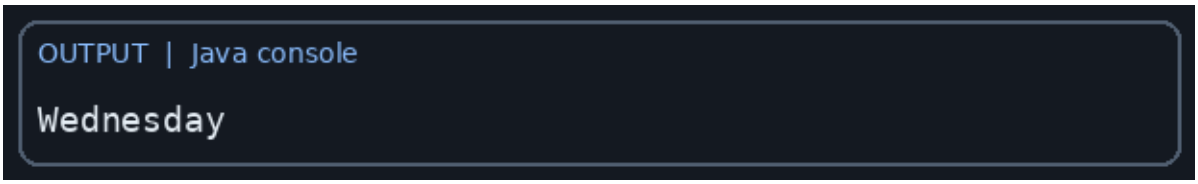
Display the day name for the selected integer.

```
public class Main {
    public static void main(String[] args) {
        int day = 3;
        switch (day) {
            case 1: System.out.println("Monday"); break;
            case 2: System.out.println("Tuesday"); break;
            case 3: System.out.println("Wednesday"); break;
            default: System.out.println("Invalid day");
        }
    }
}
```

7. Input

day = 3.

8. Output



Task 2: ArrayList basics

Add four values, then print the list, size and item at index 2.

```
import java.util.ArrayList;
public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10); list.add(20); list.add(30); list.add(40);
        System.out.println(list);
        System.out.println("Size: " + list.size());
        System.out.println("Item at index 2: " + list.get(2));
    }
}
```

7. Input

Values: 10, 20, 30, 40.

8. Output



Task 3: Update and remove ArrayList items

Change index 1 to 25 and remove the element at index 3.

```
import java.util.ArrayList;
public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10); list.add(20); list.add(30); list.add(40);
        list.set(1, 25);
        list.remove(3);
        System.out.println(list);
        System.out.println("Size: " + list.size());
    }
}
```

7. Input

Initial list: [10, 20, 30, 40].

8. Output



9. Result

The programs for Experiment 10 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.

EXPERIMENT NO. 11

Bubble Sort and Selection Sort

Student Name: _____ **Roll No.:** _____

1. Experiment Title

Bubble Sort and Selection Sort

2. Aim

Sort small arrays with bubble sort and selection sort, and count the swaps.

3. Software / Tools Required

Java JDK; Java IDE or text editor; terminal/command prompt.

4. Theory

Bubble sort repeatedly compares adjacent values and swaps them when out of order. Selection sort repeatedly selects the smallest remaining value and places it at the next sorted position. A swap exchanges two array elements.

5. Algorithm / Procedure

Implement the specified sorting method with loops, then print the sorted array or swap count.

6. Program / Queries / Commands

Task 1: Bubble sort

Sort the array in increasing order using bubble sort.

```
import java.util.Arrays;
public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 1, 4, 2, 8};
        for (int i = 0; i < arr.length - 1; i++)
            for (int j = 0; j < arr.length - 1 - i; j++)
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;
                }
        System.out.println("Sorted: " + Arrays.toString(arr));
    }
}
```

7. Input

arr = {5, 1, 4, 2, 8}.

8. Output



Task 2: Selection sort

Sort the array in increasing order using selection sort.

```
import java.util.Arrays;
public class Main {
    public static void main(String[] args) {
        int[] arr = {64, 25, 12, 22, 11};
        for (int i = 0; i < arr.length - 1; i++) {
            int min = i;
            for (int j = i + 1; j < arr.length; j++)
                if (arr[j] < arr[min]) min = j;
            int temp = arr[i]; arr[i] = arr[min]; arr[min] = temp;
        }
        System.out.println("Sorted: " + Arrays.toString(arr));
    }
}
```

7. Input

arr = {64, 25, 12, 22, 11}.

8. Output



Task 3: Count bubble-sort swaps

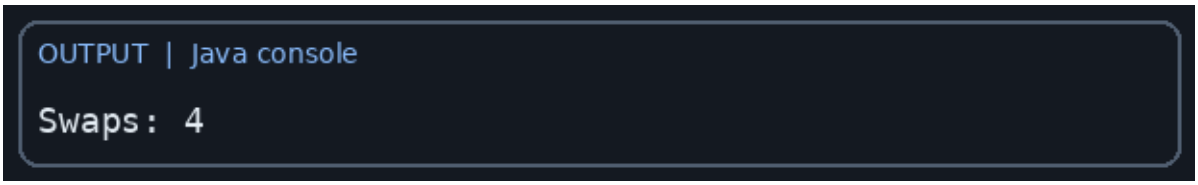
Count each adjacent exchange during bubble sort.

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 1, 4, 2, 8};
        int swaps = 0;
        for (int i = 0; i < arr.length - 1; i++)
            for (int j = 0; j < arr.length - 1 - i; j++)
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;
                    swaps++;
                }
        System.out.println("Swaps: " + swaps);
    }
}
```

7. Input

arr = {5, 1, 4, 2, 8}.

8. Output



9. Result

The programs for Experiment 11 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.

EXPERIMENT NO. 12

Insertion Sort

Student Name: _____ **Roll No.:** _____

1. Experiment Title

Insertion Sort

2. Aim

Sort small arrays with insertion sort and count how many values are shifted.

3. Software / Tools Required

Java JDK; Java IDE or text editor; terminal/command prompt.

4. Theory

Insertion sort builds a sorted portion from left to right. Each new key is compared with earlier elements, and larger elements are shifted one position right to make room for the key.

5. Algorithm / Procedure

Take each element as a key, shift larger preceding values, insert the key, and display the result or shift count.

6. Program / Queries / Commands

Task 1: Insertion sort of four values

Sort the array in increasing order.

```
import java.util.Arrays;
public class Main {
    public static void main(String[] args) {
        int[] arr = {7, 3, 5, 1};
        for (int i = 1; i < arr.length; i++) {
            int key = arr[i], j = i - 1;
            while (j >= 0 && arr[j] > key) { arr[j + 1] = arr[j]; j--; }
            arr[j + 1] = key;
        }
        System.out.println("Sorted: " + Arrays.toString(arr));
    }
}
```

7. Input

arr = {7, 3, 5, 1}.

8. Output



Task 2: Insertion sort of five values

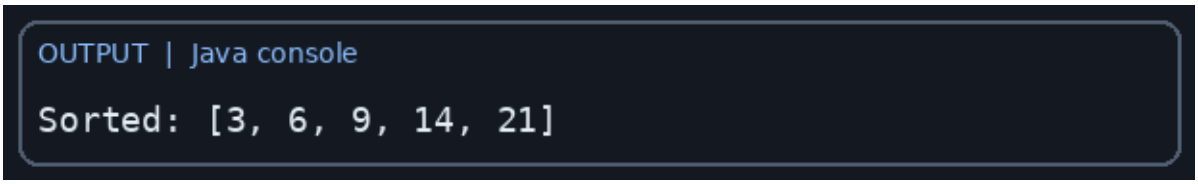
Sort the array in increasing order.

```
import java.util.Arrays;
public class Main {
    public static void main(String[] args) {
        int[] arr = {14, 3, 9, 21, 6};
        for (int i = 1; i < arr.length; i++) {
            int key = arr[i], j = i - 1;
            while (j >= 0 && arr[j] > key) { arr[j + 1] = arr[j]; j--; }
            arr[j + 1] = key;
        }
        System.out.println("Sorted: " + Arrays.toString(arr));
    }
}
```

7. Input

arr = {14, 3, 9, 21, 6}.

8. Output



Task 3: Count insertion-sort shifts

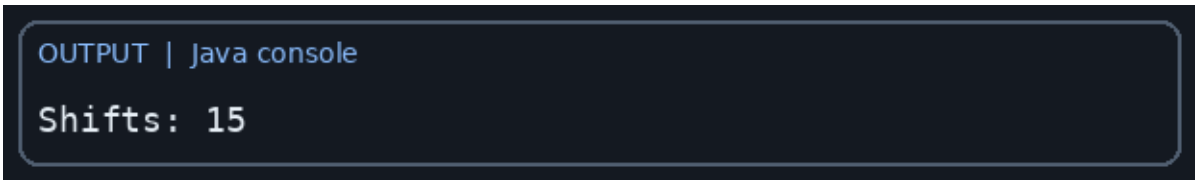
Sort the descending array and count each right shift.

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {6, 5, 4, 3, 2, 1};
        int shifts = 0;
        for (int i = 1; i < arr.length; i++) {
            int key = arr[i], j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j]; j--; shifts++;
            }
            arr[j + 1] = key;
        }
        System.out.println("Shifts: " + shifts);
    }
}
```

7. Input

arr = {6, 5, 4, 3, 2, 1}.

8. Output



9. Result

The programs for Experiment 12 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.

EXPERIMENT NO. 13

Strings: Methods and Palindromes

Student Name: _____ **Roll No.:** _____

1. Experiment Title

Strings: Methods and Palindromes

2. Aim

Use common String methods, reverse a word, check a palindrome and print substrings.

3. Software / Tools Required

Java JDK; Java IDE or text editor; terminal/command prompt.

4. Theory

String is an immutable sequence of characters. length() returns its length, indexOf() finds a character position, toUpperCase() changes letter case, charAt() accesses a character and substring() extracts part of a string. A palindrome reads the same in both directions.

5. Algorithm / Procedure

Use the requested String methods and loops to create the output.

6. Program / Queries / Commands

Task 1: String methods

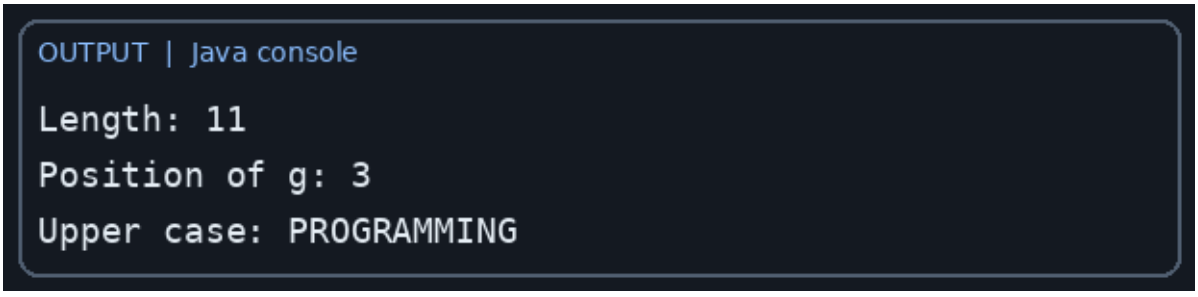
Print the length, first position of g and uppercase word.

```
public class Main {
    public static void main(String[] args) {
        String word = "Programming";
        System.out.println("Length: " + word.length());
        System.out.println("Position of g: " + word.indexOf('g'));
        System.out.println("Upper case: " + word.toUpperCase());
    }
}
```

7. Input

word = Programming.

8. Output



Task 2: Reverse a word

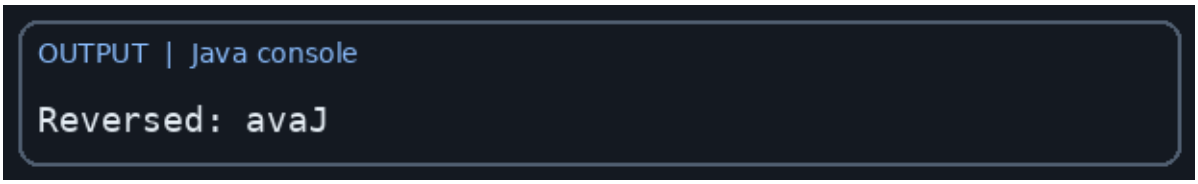
Build the reversed word one character at a time.

```
public class Main {
    public static void main(String[] args) {
        String word = "Java", reversed = "";
        for (int i = 0; i < word.length(); i++)
            reversed = word.charAt(i) + reversed;
        System.out.println("Reversed: " + reversed);
    }
}
```

7. Input

word = Java.

8. Output



Task 3: Palindrome check

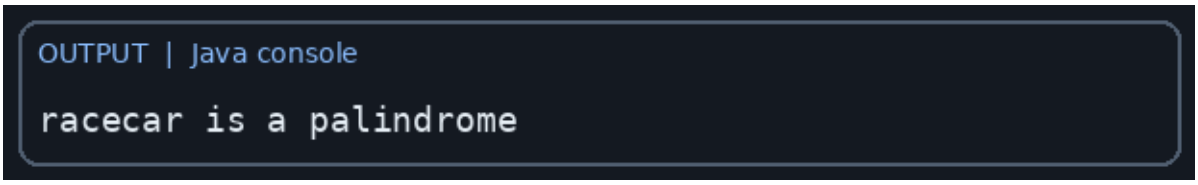
Check whether racecar reads the same from both ends.

```
public class Main {
    public static void main(String[] args) {
        String word = "racecar";
        String reversed = "";
        for (int i = word.length() - 1; i >= 0; i--) reversed += word.charAt(i);
        if (word.equals(reversed)) System.out.println(word + " is a palindrome");
        else System.out.println(word + " is not a palindrome");
    }
}
```

7. Input

word = racecar.

8. Output



Task 4: Print all substrings

Print substrings for each starting position, shortest first.

```
public class Main {
    public static void main(String[] args) {
        String word = "abc";
        for (int i = 0; i < word.length(); i++)
            for (int j = i + 1; j <= word.length(); j++)
                System.out.println(word.substring(i, j));
    }
}
```

7. Input

word = abc.

8. Output



9. Result

The programs for Experiment 13 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.

EXPERIMENT NO. 14

Characters: Counting and Positions

Student Name: _____ **Roll No.:** _____

1. Experiment Title

Characters: Counting and Positions

2. Aim

Use charAt to count vowels and a letter, and print each letter with its position.

3. Software / Tools Required

Java JDK; Java IDE or text editor; terminal/command prompt.

4. Theory

The charAt(index) method returns the character at a given zero-based position in a String. A loop can inspect every character to count vowels or occurrences of a selected letter, or print each character with its index.

5. Algorithm / Procedure

Traverse the string using a loop from index 0 to length()-1 and test each character.

6. Program / Queries / Commands

Task 1: Count vowels

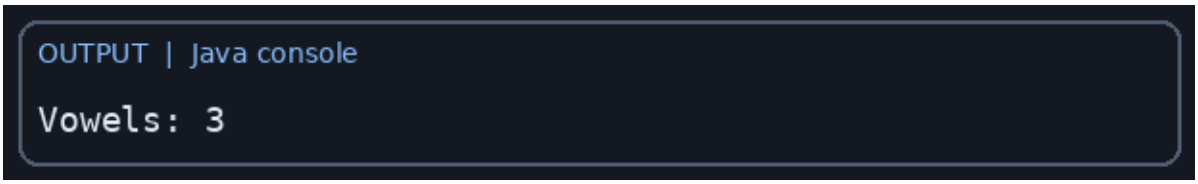
Count a, e, i, o and u in the given word.

```
public class Main {
    public static void main(String[] args) {
        String s = "programming";
        int count = 0;
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if ("aeiou".indexOf(c) >= 0) count++;
        }
        System.out.println("Vowels: " + count);
    }
}
```

7. Input

s = programming.

8. Output



Task 2: Count letter g

Count the occurrences of g in the given word.

```
public class Main {
    public static void main(String[] args) {
        String s = "programming";
        int count = 0;
        for (int i = 0; i < s.length(); i++)
            if (s.charAt(i) == 'g') count++;
        System.out.println("Count of g: " + count);
    }
}
```

7. Input

s = programming.

8. Output



Task 3: Character positions

Print every character with its zero-based index.

```
public class Main {
    public static void main(String[] args) {
        String s = "hello";
        for (int i = 0; i < s.length(); i++)
            System.out.println(i + ": " + s.charAt(i));
    }
}
```

7. Input

s = hello.

8. Output



9. Result

The programs for Experiment 14 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.

EXPERIMENT NO. 15

StringBuilder Methods

Student Name: _____ **Roll No.:** _____

1. Experiment Title

StringBuilder Methods

2. Aim

Build and change text with StringBuilder, using append, insert and deleteCharAt.

3. Software / Tools Required

Java JDK; Java IDE or text editor; terminal/command prompt.

4. Theory

StringBuilder is a mutable sequence of characters. append() adds text at the end, insert() adds text at a specified index, deleteCharAt() removes the character at an index, and length() returns the current character count.

5. Algorithm / Procedure

Create a StringBuilder, perform the requested operation and print its contents.

6. Program / Queries / Commands

Task 1: Append text and find length

Append World to Hello and print the resulting text and length.

```
public class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Hello");
        sb.append(" World");
        System.out.println(sb);
        System.out.println("Length: " + sb.length());
    }
}
```

7. Input

Initial text: Hello; appended text: World with a leading space.

8. Output



Task 2: Append numbers and remove comma

Append numbers 1 to 5 with commas, then delete the final comma.

```
public class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder();
        for (int i = 1; i <= 5; i++) sb.append(i).append(',');
        sb.deleteCharAt(sb.length() - 1);
        System.out.println(sb);
    }
}
```

7. Input

Numbers: 1 through 5.

8. Output



Task 3: Insert text at index 0

Insert Hello and a space before Java.

```
public class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Java");
        sb.insert(0, "Hello ");
        System.out.println(sb);
    }
}
```

7. Input

Initial text: Java.

8. Output



9. Result

The programs for Experiment 15 were compiled and executed successfully. The outputs shown above were obtained for the specified inputs.